

What is React?

- React is a front-end JavaScript library developed by Facebook in 2011.
- It follows the component-based approach which helps in building reusable UI components.
- It is used for developing complex and interactive web and mobile UI
- Even though it was open-sourced only in 2015, it has one of the largest communities supporting it.

What are the features of React?

- It uses the virtual DOM instead of the real DOM.
- It uses client-side rendering.
- It follows uni-directional data flow or data binding.
- It increases the application's performance.
- It can be conveniently used on the client and server side.
- Because of JSX, the code's readability increases
- React is easy to integrate with other frameworks like Meteor, Angular, etc.
- Using React, writing UI test cases becomes extremely easy.

What are the limitations of React?

- React is just a library, not a full-fledged framework
- Its library is huge and it takes time to understand.
- It can be a little difficult for novice programmers to understand
- Coding gets complex as it uses inline templating and JSX

```
import { Component } from '@angular/core';

@Component({
  selector: 'app-my-component',
  templateUrl: './my-component.component.html',
  styleUrls: ['./my-component.component.css'],
  imports: [PipeName] // need to import pipe at the top first
})
export class MyComponent {
  // Component logic goes here
}
```

What is JSX?

JSX is a shorthand for JavaScript XML. This type of file used by React utilizes the expressiveness of JavaScript along with HTML-like template syntax. This makes the HTML file easy to understand. This file makes applications robust and boosts its performance. Below is an example of JSX: JSX allows us to put HTML into JavaScript

```
function Greeting() {
  return <h1>Hello, world!</h1>;
}
```

In this example:

- The **Greeting function** is a React component.
- The return statement contains JSX, which looks like HTML but is written inside a JavaScript function.
- `<h1>Hello, world!</h1>` is the JSX code that specifies an h1 element with the text "Hello, world!".

JSX allows you to seamlessly blend HTML and JavaScript, making it easier to visualize the UI structure while leveraging JavaScript's capabilities.

What do you understand about Virtual DOM? Explain that it's working.

A virtual DOM is a lightweight JavaScript object which originally just a copy of the real DOM. It is a node tree that lists the elements, their attributes, and content as Objects and their properties. React render function creates a node tree out of the React components. It then updates this tree in response to the mutations in the data model which is caused by various actions done by the user or by the system.

How the Virtual DOM Works

1. **Initial Render:** React creates a Virtual DOM tree that mirrors the actual DOM.
2. **State Changes:** When a component's state changes, React creates a new Virtual DOM tree reflecting those changes.
3. **Diffing:** React compares the new Virtual DOM tree with the previous one to identify differences.
4. **Updating the DOM:** Only the parts of the actual DOM that have changed are updated, making the process efficient.

This approach reduces the number of direct DOM manipulations, leading to better performance and smoother user experiences.

Example of Virtual DOM Workflow

Imagine you have a simple component that displays a counter:

```

class Counter extends React.Component {
  constructor(props) {
    super(props);
    this.state = { count: 0 };
  }

  increment = () => {
    this.setState({ count: this.state.count + 1 });
  };

  render() {
    return (
      <div>
        <p>Count: {this.state.count}</p>
        <button onClick={this.increment}>Increment</button>
      </div>
    );
  }
}

```

- **Initial Render:**
 - The initial state is { count: 0 }. React creates a Virtual DOM tree with this structure and renders it to the actual DOM.
- **State Change:**
 - When the "Increment" button is clicked, this.setState({ count: this.state.count + 1 }) updates the state to { count: 1 }.
- **Virtual DOM Update:**
 - React creates a new Virtual DOM tree reflecting the updated state.
- **Diffing:**
 - React compares the new Virtual DOM tree with the previous one and detects that only the text content of the <p> element has changed (from "Count: 0" to "Count: 1").
- **DOM Update:**
 - React updates only the text content of the <p> element in the actual DOM, rather than re-rendering the entire component.

Benefits of the Virtual DOM

- **Performance:** By updating only the necessary parts of the DOM, React minimizes the number of costly DOM operations, leading to faster and more responsive applications.
- **Abstraction:** Developers work with a simple, declarative API (JSX), without worrying about the complexities of direct DOM manipulation.
- **Predictability:** The Virtual DOM ensures a consistent and predictable update process, reducing the chances of bugs and inconsistencies.

What are Props in react explain with an example.

In React, "props" (short for properties) are used to pass data from one component to another, typically from a parent component to a child component. Props are read-only, meaning they cannot be modified by the child component receiving them. They help make components reusable by allowing dynamic data to be passed and rendered.

Example of Props in React

Parent Component (App):	Child Component (Greeting):
<pre>import React from 'react'; import Greeting from './Greeting'; function App() { return (<div> <Greeting name="Alice" /> <Greeting name="Bob" /> </div>); } export default App;</pre>	<pre>import React from 'react'; function Greeting(props) { return <h1>Hello, {props.name}!</h1>; } export default Greeting;</pre>

Explanation:

- **Parent Component (App):**
 - The App component imports the Greeting component.
 - It uses the Greeting component twice, passing different values ("Alice" and "Bob") for the name prop.
- **Child Component (Greeting):**
 - The Greeting component receives props as an argument.
 - It accesses props.name to dynamically display the name passed from the parent component.

Key Points:

- Props are passed to components similarly to how attributes are passed to HTML elements.
- The child component accesses props via the props parameter.
- Props are immutable within the child component; they are meant to pass data and configuration down the component tree.

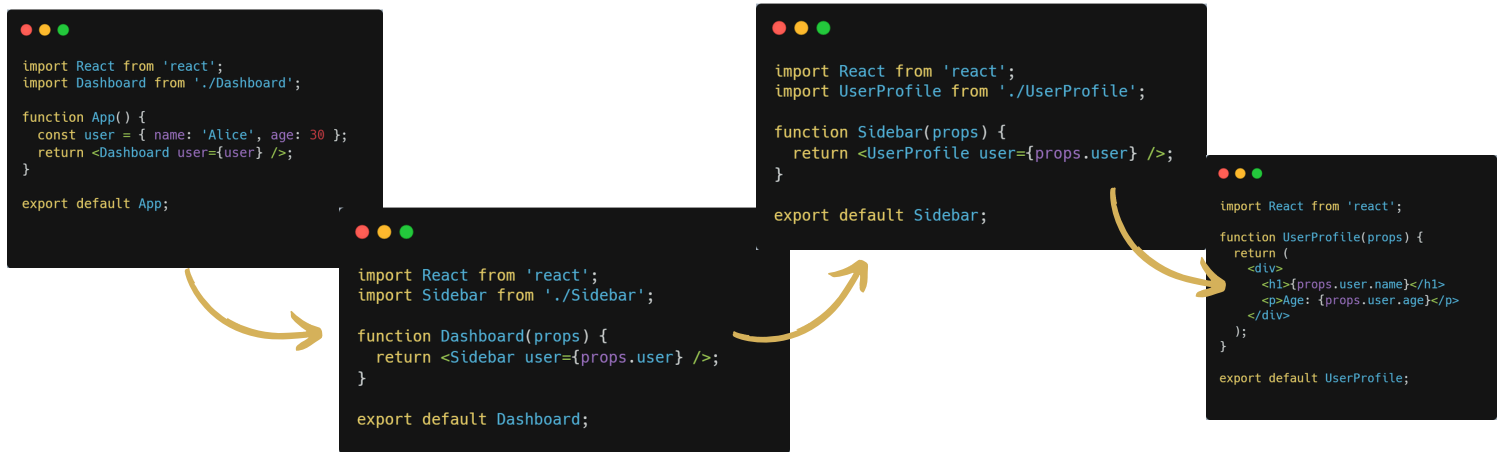
Props enable component reusability and dynamic rendering based on the data passed from parent components.

Explain what is Props Drilling

Props drilling is the process of passing data from a parent component to a deeply nested child component through multiple intermediary components.

it occurs when a parent component passes data down to its children and then those children pass the same data down to their children. This process can continue indefinitely. In the end, it's a long chain of component dependencies that can be difficult to manage and maintain.

Example of Props Drilling



Summary:

- **App** passes the user to **Dashboard**.
- **Dashboard** passes the user to the **Sidebar**.
- **Sidebar** passes the user to **UserProfile**.
- **UserProfile** displays user details.

Props drilling can be difficult to handle, especially with many nested components. Alternatives include using the Context API or state management libraries to avoid passing props through many layers.

What is a state in React?

"state" refers to a built-in object that allows components to maintain and manage dynamic data. Unlike props, which are immutable and passed from parent to child components, the state is local to the component and can change over time. When the state changes, the component re-renders to reflect the new state.

Key Characteristics of State:

- **Local to the Component:** Each component can have its state.
- **Mutable:** State can change over time, usually in response to user actions or other events.
- **Triggers Re-rendering:** When the state changes, React automatically re-renders the component to reflect the new state.
- They are accessed via **this.state()** and **updated** via **this.setState()**.

What is the arrow function in React?

Arrow functions are more of a brief syntax for writing the function expression. They are also called '**fat arrow**' (**=>**) **functions**. These functions allow binding the context of the components properly since in ES6 auto binding is not available by default. Arrow functions are mostly useful while working with higher-order functions.



```
import React from 'react';

const Greeting = () => {
  return <h1>Hello, World!</h1>;
};

export default Greeting;
```

Characteristics of Arrow Functions:

1. **Concise Syntax:** Arrow functions provide a shorter syntax compared to traditional function expressions.
2. **Lexical this Binding:** Unlike regular functions, arrow functions do not have their context; they inherit this from the parent scope.

What are the different phases of the React component's lifecycle?

There are three different phases of the React component's lifecycle:

- **Initial Rendering Phase:** This is the phase when the component is about to start its life journey and make its way to the DOM. The component is being created and inserted into the DOM.
- **Updating Phase:** Once the component gets added to the DOM, it can potentially update and re-render only when a prop or state change occurs.
- **Unmounting Phase:** This is the final phase of a component's life cycle in which the component is destroyed and removed from the DOM.

A function call happens in the below sequence:

- Parent Constructor
- Parent Render
- Child Constructor
- Child Render
- Child Component Did Mount
- Parent Component Did Mount

Explain the lifecycle methods of React components in detail?

Some of the most important lifecycle methods are:

- **componentWillMount()** - Executed just before rendering takes place both on the client as well as server-side.
- **componentDidMount()** - Executed on the client side only after the first render.

- **componentWillReceiveProps()** - Invoked as soon as the props are received from the parent class and before another render is called.
- **shouldComponentUpdate()** - Returns true or false value based on certain conditions. If you want your component to update, return true else return false. By default, it returns false. It is invoked before rendering when new props or state are being received.
- **componentWillUpdate()** - Called just before rendering takes place in the DOM.
- **componentDidUpdate()** - Called immediately after rendering takes place.
- **componentWillUnmount()** - Called after the component is unmounted from the DOM. It is used to clear up the memory spaces.

What do you understand by refs in React?

- **Refs is the shorthand for References** in React.
- It is an attribute that helps to store a reference to a particular React element or component, which will be returned by the components render configuration function.
- It is used to return references to a particular element or component returned by render().
- They come in handy when we need DOM measurements or to add methods to the components. Refs are created using `React.createRef()` method and attached to React elements via the `ref` attribute.
- To use refs throughout the component, just assign the ref to the instance property within the constructor.

```
class MyComponent extends React.Component {
  constructor(props) {
    super(props);
    this.myRef = React.createRef();
  }

  render() {
    return <div ref={this.myRef} />;
  }
}
```

List some of the cases when you should use Refs:

- Managing **focus** in input fields or other interactive elements.
- Integrating with **third-party libraries** that require direct access to DOM elements.
- **Animating** DOM elements imperatively.
- Accessing **measurements** or **positions** of DOM elements.
- **Communicating** between parent and child components when using functional components with `forwardRef`.

What is the use of forwardRef?

`forwardRef` is a higher-order component (HOC) or a technique in React that allows a component to pass a ref down to one of its children. This is particularly useful when you want to access a child component's DOM node or React component instance from its parent component.

Use Cases for forwardRef:

- **DOM Access:** When you need to access a DOM element within a child component from its parent component.
- **Imperative Component Communication:** When you want to call methods or access properties of a child component directly from its parent component.
- `forwardRef` is used to **pass a ref from a parent component** to a child component.
- It enables **parent components** to access the underlying **DOM nodes** or React component instances of their children.
- **`forwardRef`** is particularly useful when working with components that need to be imperatively controlled or accessed by their parent components.

Controlled and Uncontrolled Components?

Controlled components refer to components that have their state and behavior controlled by the parent component. These components rely on props passed down from the parent component to update their state and behavior. Uncontrolled components refer to components that manage their state internally. They are controlling itself.

Key Differences:

- **State Handling:** Controlled components manage form data using React state, while uncontrolled components rely on direct DOM manipulation.
- **Validation:** Controlled components allow for easier validation and manipulation of form data before submission.
- **Complexity:** Controlled components may require more code to manage state and handle changes, while uncontrolled components may be simpler but require more direct DOM interaction.

Choosing Between Controlled and Uncontrolled Components:

- **Controlled Components:** Prefer controlled components when you need more control over form data, such as validation and manipulation.
- **Uncontrolled Components:** Use uncontrolled components when integrating with legacy code or third-party libraries that require direct DOM access. They can also be simpler for basic form handling without complex validation requirements.

What is Lifting State Up in react?

Lifting state up is a technique used in React to share state between multiple components. Instead of each component having its own local state, the state is lifted up to their closest common ancestor. This common ancestor then passes the state down to the components through props. Sometimes, you want the state of two components to always change together. To do it, remove the state from both of them, move it to their closest common parent, and then pass it down to them via props. This is known as lifting state up, and it's one of the most common things you will do when writing React code

What are Higher Order Components(HOC)?

Higher Order Components (HOCs) in React are advanced techniques for reusing component logic. An HOC is a function that takes a component and returns a new component with added functionality. Here's a concise overview:

1. **Definition:** HOCs are functions that wrap a component to provide additional props or functionality.
2. **Purpose:** They allow you to reuse logic across multiple components without repeating code.

3. **Usage:** They are often used for cross-cutting concerns like logging, authentication, or data fetching.

HOC can be used for many tasks like

- Code reuse, logic, and bootstrap abstraction
- Render Hijacking
- State abstraction and manipulation
- Props manipulation

What are Pure Components in react?

In React, a Pure Component is a component that renders the same output for the same props and state, helping to optimize performance by preventing unnecessary re-renders.

1. **Definition:** Pure Components are components that implement a shallow comparison on props and state in `shouldComponentUpdate`.
2. They improve performance by avoiding unnecessary re-renders when the props and state haven't changed.

```
import React, { PureComponent } from 'react';

class MyComponent extends PureComponent {
  render() {
    console.log('Rendering MyComponent');
    return <div>{this.props.value}</div>;
  }
}

// Usage
<MyComponent value="Hello" />
```

What is the significance of keys in React?

In React, keys are used to identify elements in a list. They help React efficiently update and render elements by providing a stable identity for each element. Here's a brief explanation of their significance:

Significance of Keys:

1. **Efficient Updates:** Keys help React identify which items have changed, been added, or removed. This allows React to optimize the rendering process and only update the parts of the DOM that have changed.
2. **Stable Identity:** Keys give elements a stable identity, ensuring that they are preserved between renders. This is especially important when dynamically creating lists of elements.
3. **Preventing Re-renders:** Proper use of keys can prevent unnecessary re-renders and reordering of elements, improving performance.

```
function ItemList({ items }) {
  return (
    <ul>
      {items.map(item => (
        <li key={item.id}>{item.name}</li>
      ))}
    </ul>
  );
}
```

In this example, each `li` element has a **unique key** based on `item.id`. This helps React efficiently update the list when items change, ensuring optimal **performance** and a **stable UI**.

What is a React Router?

React Router is a standard library for routing in React applications. It enables the navigation among views of various components in a React application, allows changing the browser URL, and keeps the UI in sync with the URL.

Key Features:

1. **Declarative Routing:** Define routes in a declarative manner using JSX.
2. **Dynamic Routing:** Routes can change based on app state and URL parameters.
3. **Nested Routing:** Support for nested routes, allowing complex UI structures.
4. **URL Parameters:** Allows for URL parameters to pass data to components.

Explanation:

1. **Router:** `BrowserRouter` is used to wrap the entire application, enabling routing.
2. **Link:** `Link` components are used to create navigation links without reloading the page.
3. **Route:** `Route` components define which components to render based on the current URL path.
4. **Switch:** `Switch` is used to render only the first matching route, ensuring only one component renders at a time for a given URL.

React Router provides a simple and powerful way to manage routing in React applications, making it easier to create dynamic and interactive web apps.

Why is the `switch` keyword used in React Router v4?

In React Router v4, the `Switch` component ensures that only the first matching `Route` is rendered. This prevents multiple routes from rendering at the same time. It is particularly useful for implementing exclusive routing and handling 404 pages.

```
import { BrowserRouter as Router, Route, Switch } from 'react-router-dom';

function App() {
  return (
    <Router>
      <Switch>
        <Route exact path="/" component={Home} />
        <Route path="/about" component={About} />
        <Route component={NotFound} /> {/* Fallback for unmatched routes */}
      </Switch>
    </Router>
  )
}
```

- **exact Prop:** Ensures the `Home` component only matches the exact path `/`.
- **Fallback Route:** The last `Route` without a path catches all unmatched paths, rendering a 404 component.

What is `<HashRouter>` ?

In React Router, `HashRouter` is a type of router that uses the hash portion (`#`) of the URL for

routing. It's useful when server-side configuration for route handling isn't possible or needed. The URLs look like `http://example.com/#/about`, and it's straightforward to implement in environments where full HTML5 history API support isn't available.

What is Exact Route in react?

In React Router, the `exact` prop is used to ensure that a route matches the URL path exactly, rather than just partially matching. This is particularly useful when you have multiple routes and want to render a specific component only when the path matches exactly.

```
import { BrowserRouter as Router, Route } from 'react-router-dom';

function App() {
  return (
    <Router>
      <div>
        <Route exact path="/" component={Home} />
        <Route path="/about" component={About} />
      </div>
    </Router>
  );
}
```

- **exact Prop:** In the above example, `<Route exact path="/" component={Home} />` ensures that the Home component is only rendered when the URL path is exactly `/`. Without `exact`, `/about` would also match `/`, since `/` is a prefix of `/about`.

Uses:

1. **Home Page:** Typically used for the root route of your application to render a specific component only when the URL is exactly `/`.
2. **Navbar Links:** Ensures that navigation links in a navbar highlight correctly when the exact path matches.

Note:

Always use `exact` on the route that should match only the exact path to avoid unexpected rendering of components. This helps maintain the expected UI behavior based on the URL path.

Why do we need a Router in React?

A Router is used to define multiple routes and when a user types a specific URL, if this URL matches the path of any 'route' defined inside the router, then the user is redirected to that particular route. So basically, we need to add a Router library to our app that allows creating multiple routes with each leading to us a unique view. `import { Route } from "react-router-dom";`

```
<switch>
  <route exact path={ } component={Home}/>
  <route path={ } component={Newpost}/>
  <route path={ } component={Post}/>
</switch>
```

What will happen if we write `setState()` inside a `componentWillMount()` ?

`componentWillMount()` is invoked immediately before mounting occurs. It is called before `render()`, therefore setting the state in this method will not trigger a re-render. Avoid introducing any side effects or subscriptions in this method

What will happen if we write `setState()` inside a `render()` ?

Calling `setState()` here makes your component a contender for producing infinite loops. The `render()` function should be pure, meaning that it does not modify the component state, it returns the same result each time it's invoked, and it does not directly interact with the browser. In this case, avoid using `setState()` here.

What are Error Boundaries?

Error Boundaries are React components that catch JavaScript errors anywhere in their child component tree, log those errors, and display a fallback UI. Components become Error Boundaries if they define either: `getDerivedStateFromError()` or `componentDidCatch()`.

Error Boundaries are usually defined once and used throughout the application by wrapping around other components like catch blocks.

Client-side rendering & Server-side rendering?

	Client-Side Rendering (CSR)	Server-Side Rendering (SSR)
Rendering Location	Renders the UI in the browser using JavaScript.	Renders the UI on the server and sends the HTML to the client.
Initial Load Time	Slower initial load, as the entire JavaScript bundle must be downloaded and executed.	Faster initial load, as the server sends pre-rendered HTML to the browser.
Subsequent Interactions	Faster, as the UI is already loaded and React handles updates efficiently in the browser.	Slower for subsequent interactions if re-fetching data or doing a full-page reload is required.
SEO	Poor, as web crawlers may struggle with JavaScript-rendered content (though improvements like pre-rendering help).	Better, as content is fully rendered in HTML before reaching the client, making it easier for crawlers to index.
Performance	Better for highly interactive applications that need real-time updates without full page reloads.	Better for static or less dynamic content, as the server handles heavy lifting before sending content.
JavaScript Dependency	High dependency on JavaScript to render content.	Less dependent, as HTML content is already available when sent from the server.
Caching	Challenging, as content is dynamically generated on the client.	Easier, as static HTML can be cached at various levels (CDN, browser, etc.).
User Experience (UX)	May experience a delay before seeing any content (blank page) until JavaScript is fully loaded.	Faster perceived load, as the user sees content sooner (HTML arrives pre-rendered).
Development Complexity	Easier to set up, as React operates solely on the client.	More complex, as the server and client must both handle rendering logic (Next.js or similar frameworks are often used).
Best Use Cases	Applications with heavy user interaction and dynamic updates (e.g., dashboards, SPAs).	Content-heavy sites, blogs, or e-commerce platforms where SEO and fast load times are critical.

Lazy Component in react

`lazy()` It is a new function in react that lets you load react components lazily through code splitting without help from additional libraries. Lazy loading is the technique of rendering only-needed or critical user interface items first, then quietly unrolling the non-critical items later. Lazy loading is a design pattern. It allows you to load parts of your application on-demand to reduce the initial load time.

Webpack

Webpack is a popular module bundling system built on top of Node.js. It can handle not only the combination and minification of JavaScript and CSS files, but also other assets such as image files (sprising) through plugins. Webpack is a tool that lets you compile JavaScript modules, also known as module bundler. Given many files, it generates a single file (or a few files) that run your app. It can perform many operations: helps you bundle your resources.

React Hooks Guide

useState

Allows you to add state to functional components.

Syntax:

```
const [state, setState] = useState(initialState);
```

Example:

```
import React, { useState } from 'react';

function Counter() {
  const [count, setCount] = useState(0);

  return (
    <div>
      <p>You clicked {count} times</p>
      <button onClick={() => setCount(count + 1)}>
        Click me
      </button>
    </div>
  );
}
```

2. useEffect

Handles side effects in function components (e.g., data fetching, subscriptions).

```

useEffect(() => {
  // side effect
  return () => {
    // cleanup
  };
}, [dependencies]); // Optional array of dependencies

import React, { useState, useEffect } from 'react';

function DataFetcher() {
  const [data, setData] = useState(null);

  useEffect(() => {
    fetch('https://api.example.com/data')
      .then(response => response.json())
      .then(data => setData(data));
  }, []); // Empty array = run once after initial render

  return <div>Data: {JSON.stringify(data)}</div>;
}

```

useContext

Allows access to context values without needing Consumer.

Syntax: `const value = useContext(MyContext);`

```

import React, { useContext } from 'react';
const MyContext = React.createContext();

function Component() {
  const value = useContext(MyContext);
  return <div>{value}</div>;
}

function App() {
  return (
    <MyContext.Provider value="Hello, World!">
      <Component />
    </MyContext.Provider>
  );
}

```

useReducer

A more powerful alternative to useState, often used for complex state logic.

Syntax: `const [state, dispatch] = useReducer(reducer, initialState);`

```

const initialState = { count: 0 };

function reducer(state, action) {
  switch (action.type) {
    case 'increment':
      return { count: state.count + 1 };
    case 'decrement':
      return { count: state.count - 1 };
    default:
      throw new Error();
  }
}

function Counter() {
  const [state, dispatch] = useReducer(reducer, initialState);

  return (
    <div>
      <p>Count: {state.count}</p>
      <button onClick={() => dispatch({ type: 'increment' })}>
        +
      </button>
      <button onClick={() => dispatch({ type: 'decrement' })}>
        -
      </button>
    </div>
  );
}

```

useRef

Provides a way to persist values across renders without causing a re-render. Often used to reference DOM elements.

Syntax: `const refContainer = useRef(initialValue);`

Example:

```

import React, { useRef, useEffect } from 'react';

function InputFocus() {
  const inputEl = useRef(null);

  useEffect(() => {
    inputEl.current.focus(); // Automatically focus input on render
  }, []);

  return <input ref={inputEl} type="text" />;
}

```

useMemo

Memoizes a computed value to optimize performance by avoiding expensive calculations on every render.

Syntax: `const memoizedValue = useMemo(() => computeExpensiveValue(a, b), [a, b]);`

```
import React, { useMemo, useState } from 'react';

function ExpensiveComponent({ num }) {
  const [multiplier, setMultiplier] = useState(1);

  const expensiveCalculation = useMemo(() => {
    return num * 1000;
  }, [num]);

  return (
    <div>
      <p>Expensive result: {expensiveCalculation}</p>
      <button onClick={() => setMultiplier(multiplier + 1)}>Multiply</button>
    </div>
  );
}
```

useCallback

Returns a memoized version of a callback function, useful for passing stable functions to child components to avoid unnecessary re-renders.

Syntax:

`const memoizedCallback = useCallback(() => {`

`// callback logic`

`}, [dependencies]);`

Example:

```
import React, { useState, useCallback } from 'react';

function Child({ onClick }) {
  return <button onClick={onClick}>Click Me</button>;
}

function Parent() {
  const [count, setCount] = useState(0);

  const handleClick = useCallback(() => {
    setCount(count + 1);
  }, [count]);

  return (
    <div>
      <Child onClick={handleClick} />
      <p>Count: {count}</p>
    </div>
  );
}
```

Rules of Hooks

- Only Call Hooks at the Top Level

Don't call Hooks inside loops, conditions, or nested functions.

- Only Call Hooks in React Functions

Hooks can only be used in functional components or custom Hooks.

Custom Hooks

Custom Hooks let you extract and reuse logic from components.

Example:

```

function useWindowWidth() {
  const [width, setWidth] = useState(window.innerWidth);

  useEffect(() => {
    const handleResize = () => setWidth(window.innerWidth);
    window.addEventListener('resize', handleResize);
    return () => window.removeEventListener('resize', handleResize);
  }, []);

  return width;
}

function Component() {
  const width = useWindowWidth();
  return <p>Window width: {width}px</p>;
}

```

Some Rapid fire question for your

1. What are React Hooks? Why were they introduced?

Hooks are functions that let you use React state and lifecycle features in functional components. They were introduced to simplify code, reduce the reliance on class components, and allow functional components to use state and other React features.

2. What is the useState hook and how does it work? Can you provide an example?

useState is a Hook that allows you to add state to functional components. It returns an array with two values: the current state and a function to update it.

```
const [count, setCount] = useState(0);
```

3. What is the useEffect hook? What are its typical use cases?

useEffect runs side effects in functional components. It can mimic lifecycle methods like componentDidMount, componentDidUpdate, and componentWillUnmount.

Common use cases include fetching data, manually updating the DOM, or setting up subscriptions.

4. Explain the difference between useEffect and useLayoutEffect.

useEffect runs asynchronously after the render is painted, while useLayoutEffect runs synchronously after all DOM mutations but before the browser repaints.

5. How does React useContext work, and when would you use it?

useContext allows you to subscribe to React context without needing a Consumer component. It's used when you need to access and share data across components without passing props.

6. Explain how useRef works. What are some common use cases for it?

useRef returns a mutable object with a .current property. It persists across renders without triggering re-renders.

Common uses: Accessing DOM elements directly and storing mutable values like timers.

7. What is the useReducer hook and how is it different from useState?

useReducer is used to manage complex state logic and is an alternative to useState. It takes a reducer function and an initial state and returns the current state and a dispatch function.

It's useful for state logic that involves multiple sub-values or complex transitions.

8. When would you use useMemo and how does it optimize performance?

useMemo memoizes a computed value to prevent expensive calculations on every render. It only recalculates the value when its dependencies change.

javascript

```
const memoizedValue = useMemo(() => computeExpensiveValue(a, b), [a, b]);
```

9. What is useCallback and how does it work with memoization?

useCallback memoizes a function so it's not recreated on every render unless its dependencies change. It's particularly useful when passing callbacks to child components that rely on referential equality.

10. Can you explain how the dependency array works in useEffect? What happens if you leave it empty?

The dependency array determines when the useEffect hook runs. If left empty, useEffect runs only once on mount and cleanup on unmount.

11. What are the potential pitfalls of using useEffect, and how can you avoid them?

Pitfalls include missing dependencies in the dependency array or triggering an infinite loop. Always include dependencies or use the useCallback and useMemo hooks to prevent such issues.

12. How would you mimic componentDidMount, componentDidUpdate, and componentWillUnmount using hooks?

componentDidMount: useEffect with an empty dependency array.

componentDidUpdate: useEffect with dependencies.

componentWillUnmount: return a cleanup function from useEffect.

13. What is the purpose of custom hooks? Can you create one during the interview?

Custom hooks allow you to reuse stateful logic between components.

Example: javascript



```
function useWindowWidth() {
  const [width, setWidth] = useState(window.innerWidth);
  useEffect(() => {
    const handleResize = () => setWidth(window.innerWidth);
    window.addEventListener('resize', handleResize);
    return () => window.removeEventListener('resize', handleResize);
  }, []);
  return width;
}
```

14. How can you handle API calls with `useEffect` to avoid multiple requests?

Use a cleanup function in `useEffect` to cancel ongoing requests if the component unmounts or dependencies change.

15. What is the difference between `useMemo` and `useCallback`? Can you provide scenarios where each is more appropriate?

`useMemo` is used to memoize values, while `useCallback` memoizes functions. Use `useMemo` for expensive calculations and `useCallback` for functions passed as props to child components.

16. How would you handle state that is shared between multiple components using hooks?

You can use `useContext` to share state globally, or pass down state and update functions as props.

17. Explain the concept of custom hooks. When would you create one?

Custom hooks encapsulate reusable logic, allowing it to be shared across multiple components. Create one when you find yourself repeating logic.

18. Can you create a custom hook to fetch data from an API and demonstrate its use in a component?

Example custom hook for fetching data:



```
function useFetch(url) {
  const [data, setData] = useState(null);
  const [error, setError] = useState(null);
  useEffect(() => {
    fetch(url)
      .then((res) => res.json())
      .then(setData)
      .catch(setError);
  }, [url]);
  return { data, error };
}
```

19. What are the best practices for naming custom hooks?

Custom hooks should always start with the prefix "use" to follow conventions and help React's linting tools detect correct usage.

20. How would you structure a custom hook to manage form handling and validation?

Create a hook that manages form state, handles input changes, and validates the form. Example

```
function useForm(initialValues, validate) {
  const [values, setValues] =
  useState(initialValues);
  const [errors, setErrors] = useState({});
  const handleChange = (e) => {
    setValues({
      ...values,
      [e.target.name]: e.target.value,
    });
  };
  const handleSubmit = (e) => {
    e.preventDefault();
    setErrors(validate(values));
  };
  return {
    values,
    errors,
    handleChange,
    handleSubmit
  };
}
```

21. Explain the compound component pattern and how it can be implemented using hooks.

The compound component pattern allows components to work together seamlessly by sharing state. Hooks like useContext can be used to share state among compound components.

22. How would you approach managing complex state with useReducer in a real-world application?

Use useReducer when state logic involves multiple sub-values or actions. For large applications, combine it with context for better state management.

23. Have you encountered stale closure issues when using hooks? How did you resolve them?

Stale closure issues occur when a hook references outdated values. To resolve, ensure dependencies are correctly added to the hook's dependency array or use refs to store values.

24. How would you debounce a search input using hooks?

Debouncing can be achieved by using useEffect with a timer to delay the update function:



```
function useDebounce(value, delay) {  
  const [debouncedValue, setDebouncedValue] = useState(value);  
  useEffect(() => {  
    const handler = setTimeout(() => setDebouncedValue(value),  
delay);  
    return () => clearTimeout(handler);  
  }, [value, delay]);  
  return debouncedValue;  
}
```

25. Can you explain how hooks work under the hood? How does React keep track of hook calls between renders?

React uses a linked list to store hooks internally. Each hook call is tracked by an index, ensuring the same hook is called in the same order across renders.

26. How would you optimize performance in a large component using `useMemo`, `useCallback`, and `React.memo`?

Use `useMemo` to memoize expensive calculations, `useCallback` to memoize functions, and `React.memo` to prevent unnecessary re-renders for child components.

27. What are some of the limitations or drawbacks of hooks that you've encountered in your projects?

Hooks can introduce complexity in large components, make debugging harder, and are prone to issues like stale closures and dependency array mismanagement.

Check the practical React interview questions as well in another document

To enhance your learning, I invite you to check out my YouTube channel: [UI Dev Guide](#). The channel is dedicated to providing mock interviews for job seekers, helping in [career consulting](#), and practical examples that mostly asked in the interviews check below playlist for javascript angular and react:

Angular : 🌐 Must Watch Angular interviews

React: 🌐 Must watch React interviews list before going for interview

Javascript Coding :

🌐 JavaScript coding interview questions | JavaScript interview questions and answers

Javascript: 🌐 JavaScript interview questions